When using PostgreSQL with Active Record, an optimistic locking technique can be used. This technique can be used when multiple transactions are updating the same row.

With optimistic locking, a lock is taken "late." Each process with an optimistic lock can check whether the object being locked has already been modified. When the second modification has lost the "race" to modify the locked object, an ActiveRecord::StaleObjectError exception is raised.

To use this functionality, add a lock_version column to tables used by Active Record objects that you wish to lock. The column is automatically updated by Active Record when objects are instantiated.

The Engine Yard post *A Guide to Optimistic Locking*[24] shows how to handle concurrent modifications. When locking is required for the application, and the locking experience involves user interface level interaction, consider rescuing this error, and rendering the edit page with the latest version of the object instantiated. Add a helpful Rails *Flash Message* for the user, to describe that their edit was not allowed.

When locking is behind the scenes and not user facing, lock management at the row level may best be done using the PostgreSQL capabilities of SELECT with either FOR UPDATE or FOR SHARE. Use FOR UPDATE when the goal is to select some rows and prevent concurrent updates.

What about lighter weight locks?

Using Advisory Locks

PostgreSQL provides a weaker lock type called an *Advisory Lock*. Advisory Locks in Active Record use the Ruby standard library Mutex.

Ruby on Rails uses Advisory Locks. One example is when Active Record Migrations are run, an Advisory Lock is taken to prevent two threads of execution from applying the same migration. If this happened, an ActiveRecord::ConcurrentMigrationError error is raised.

Advisory Locks are useful, but come with other trade-offs. The Transaction Pooling Mode for PgBouncer cannot be used with Advisory Locks. To use Transaction Pooling Mode with PgBouncer, from Rails 6, Advisory Locks can be disabled[25] by setting advisory_locks: false in config/database.yml. As you saw earlier, you'd also need to disable Prepared Statements.

24. https://www.engineyard.com/blog/a-guide-to-optimistic-locking/
25. https://blog.saeloun.com/2019/09/09/rails-6-disable-advisory-locks.html